1. **Aim/Overview of the practical:** Write a program to Implementing Naïve Bayes algorithm.
   Define any problem which can be solved using Naïve Bayes algorithms.
   Specify datasets for your problem.
   Execute and discuss the outcomes of the experiment.

   b) Implementation of Decision Tree using Panda Libraries and Graph.
   Creating a Model
   Training a Model
   Calculating Ginn Index (GI) and Gain
   Design a Decision Tree

2. **Objective of the practical:** To understand and implement classification techniques using Naïve Bayes and Decision Tree algorithms, and analyze their performance on real-world data.

3. **Code for experiment/practical:**
   Naïve Bayes

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# Step 1: Load dataset
data = pd.read_csv("diabetes_dataset.csv")

# Step 2: Features & Target
X = data.drop("Outcome", axis=1)
```

```
y = data["Outcome"]

# Step 3: Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 4: Train Naive Bayes model
model = GaussianNB()
model.fit(X_train, y_train)

# Step 5: Predictions
y_pred = model.predict(X_test)

# Step 6: Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Step 7: Confusion Matrix plot
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=model.classes_)
disp.plot(cmap="Blues")
plt.title("Naive Bayes - Diabetes Dataset (Confusion Matrix)")
plt.show()
```

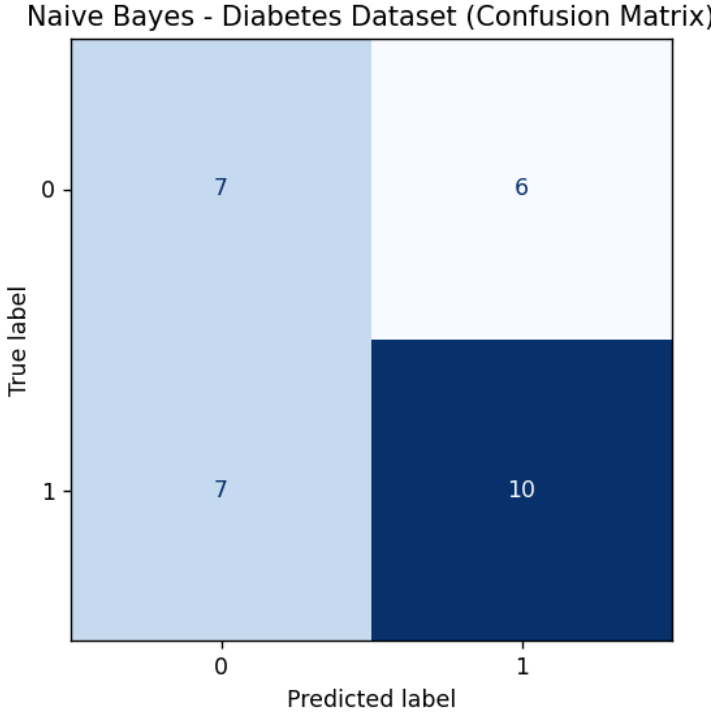
Decision Tree

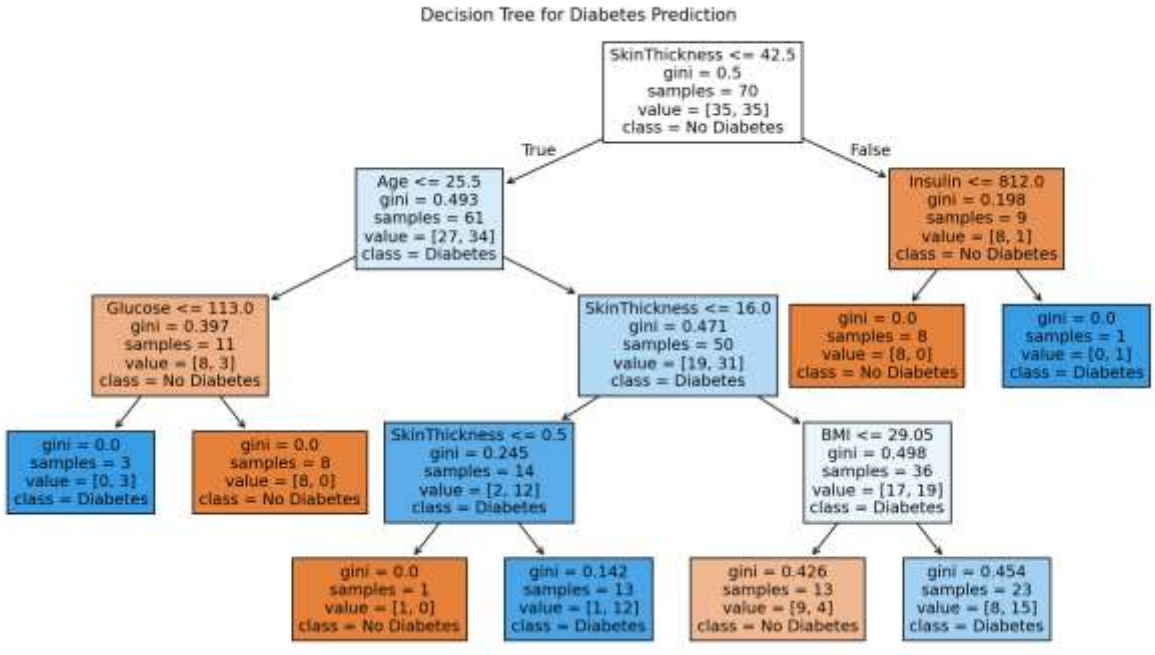
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Step 1: Read dataset from CSV
df = pd.read_csv("diabetes_dataset.csv")

print("Dataset Preview:")
print(df.head())

# Step 2: Creating Model (Features and Target)
X = df.drop('Outcome', axis=1)
y = df['Outcome']
```

```
# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 3: Training a Model
model = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
model.fit(X_train, y_train)

# Step 4: Prediction & Accuracy
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"\nModel Accuracy: {accuracy * 100:.2f}%")

# Step 5: Calculating Gini Index Example
def gini_impurity(p):
    return 1 - (p**2 + (1-p)**2)

p = y_train.mean()
gini = gini_impurity(p)
print(f"Gini Impurity Example (for target proportion {p:.2f}): {gini:.3f}")

# Step 6: Visualize Decision Tree
plt.figure(figsize=(15,8))
plot_tree(model, filled=True, feature_names=X.columns, class_names=['No Diabetes',
'Diabetes'])
plt.title("Decision Tree for Diabetes Prediction")
plt.show()
```

4. **Output:**

Naïve Bayes





5. Learning Outcomes (What I have learnt):

- Learned how to preprocess and prepare datasets for ML models.
- Understood the working and applications of Naïve Bayes classification.
- Implemented and visualized a Decision Tree using Pandas and Graph libraries.
- Analyzed model accuracy and interpreted results based on Gini Index and Gain.